

BOS: Bottleneck-Oriented Overlap Scheduling on Identical Parallel Machines

Chen-Hong

Department of Computer Science and Information Engineering
National Central University, Taoyuan, Taiwan
leon28473787@gmail.com

Abstract—We study non-preemptive makespan minimization on identical parallel machines with release dates and edge-specific minimum start–start lags. For an edge (i, j) , task j may start after task i has processed for $\ell_{ij} = p_i - \delta_{ij}$ time units; the overlap allowance δ_{ij} changes temporal feasibility but not either task’s processing requirement. We propose BOS, an event-driven list scheduler whose fixed offline priority is the maximum overlap-adjusted downstream path length. A single realized blocking chain yields $C_{\max} \leq W/m + (1 - 1/m)L_\delta$ when all releases are zero and $C_{\max} \leq W/m + L_{r,\delta}$ otherwise. Hence every work-conserving list scheduler under this model, including BOS, is a $(2 - 1/m)$ -approximation in the zero-release case and a 2-approximation with arbitrary finite releases.

We also provide a schedule-induced constraint-DAG diagnostic and a fully reproducible Python implementation. The evaluation compares six list priorities on 5,760 synthetic main instances and 576 independently certified small-instance optima. On nontrivial positive-overlap instances, BOS improves mean makespan over a HEFT-derived rank by only 0.188% with zero releases and 0.140% with uniform releases. On the 432 nontrivial exact instances, their mean optimality gaps are 1.783% and 1.809%, respectively; GRPW obtains 1.803%. These results reject a dominance claim: BOS’s defensible contribution is an overlap-aware priority embedded in a model with explicit guarantees, diagnostics, and reproducible evidence.

Index Terms—precedence-constrained scheduling, identical parallel machines, start–start time lags, list scheduling, reproducible experiments.

I. INTRODUCTION

The classical problem $P \mid \text{prec} \mid C_{\max}$ assigns non-preemptive precedence-constrained tasks to identical parallel machines and minimizes the last completion time [1], [2]. Its usual finish–start semantics require a predecessor to complete before a successor starts. Some manufacturing and project workflows instead permit a successor to start after a prescribed prefix of each predecessor has executed. Examples include staged transfer, progressive handoff, and overlapped assembly. A minimum start–start lag is a direct abstraction of this timing rule.

This paper considers a fixed directed acyclic graph (DAG). Every task, processing time, release date, and lag is known before scheduling. The setting is therefore *offline*; event-driven execution describes how the ready set is maintained, not uncertainty about future arrivals. Dependencies also retain AND semantics: a successor must satisfy the lag on every incoming edge. Thus the model does not represent an OR choice or a policy that accepts an arbitrary subset of components.

The overlap rule creates a proof obligation that is easy to miss. Classical list-scheduling intuition says that non-full machine time is caused by a precedence chain, but independently summing every idle or overlap interval can charge mutually incompatible paths. The valid argument must trace one schedule-dependent blocking chain from a last-finishing task and charge disjoint prefixes of that chain. This distinction is central to the approximation proof below.

We study BOS (Bottleneck-Oriented Overlap Scheduling), a deterministic list rule. For a fixed instance, BOS assigns each task the maximum overlap-adjusted length of a path beginning at that task. The score is static; the dispatch decision changes because release and edge activation events change the executable set. After a schedule is constructed, a second DAG augmented with realized machine-order arcs identifies whether its critical chain is driven by releases, precedence, or resource competition.

The contributions are as follows.

- We state an identical-machine scheduling model with edge-specific minimum start–start lags and give a complete, implementable event semantics, including simultaneous zero-lag activation.
- We prove bottom-level correctness, schedule feasibility, termination, and $O((|V|+|E|)\log(|V|+|E|))$ implementation time. A single blocking chain yields a $(2 - 1/m)$ approximation for zero releases and a release-aware factor of two.
- We separate what the theorem does and does not establish. The approximation proof uses work conservation rather than the BOS priority and consequently applies to every work-conserving list order under the same model.
- We provide executable Python code, unit tests, raw instance results, a time-indexed exact model, and regenerated figures. Six priority policies are evaluated on 5,760 main instances and 576 certified optima, followed by a 4,000-instance contrast search and a scaling study.

The evidence supports a deliberately narrow claim. BOS and a HEFT-derived rank usually produce the same makespan and neither dominates the other. BOS is useful as an overlap-aware, auditable priority within a rigorously specified scheduling framework; it is not presented as a universal replacement for established heuristics.

II. RELATED WORK AND POSITIONING

Graham's list-scheduling analysis gives the classical $(2 - 1/m)$ bound for precedence-constrained jobs on identical processors [2]. General temporal constraints, including minimum and maximum time lags, have also long appeared in project scheduling and RCPSP variants [3], [4]. Consequently, an edge-wise start–start lag is not by itself a novelty claim. Our narrower contribution is a self-contained charging proof for the present identical-machine specialization, where the lag is bounded by the predecessor processing time, together with an overlap-adjusted list priority and a reproducible evaluation.

HEFT was proposed for DAG scheduling on heterogeneous processors and combines an upward rank with processor selection by earliest finish time [5]. On identical processors with zero communication cost, its upward rank reduces to a full-processing-time downstream path score. Our comparator uses that rank inside the same event-driven feasibility engine as BOS. We call it *HEFT-rank* rather than full HEFT because processor heterogeneity and communication-aware selection are absent. This controlled construction isolates the effect of replacing full-work path weights by overlap-adjusted lags.

Parallel schedule-generation schemes and priority rules are long-standing RCPSP tools [6]. We therefore include duration rules and GRPW in addition to HEFT-rank and FIFO. Exact methods for generalized precedence variants provide a further reference point for small-instance validation [7].

Resource-constrained project scheduling studies richer combinations of renewable resources, modes, and generalized precedences [4], [8]–[10]. In comparison, every task here requires exactly one unit from a pool of m identical machines, but the restricted structure makes an explicit list-scheduling bound and near-linear event simulation possible. The schedule-induced graph is related to disjunctive and resource-flow views of a realized schedule; we use it as a certificate and diagnostic rather than as a claim that the fixed input DAG itself changes.

III. MODEL AND LOWER BOUNDS

Let $G = (V, E)$ be a DAG. Task $i \in V$ has processing time $p_i > 0$ and finite release date $r_i \geq 0$. There are m identical machines. Each task uses one machine, runs non-preemptively, and is assigned a start time s_i and machine $\mu_i \in \{1, \dots, m\}$.

For edge $(i, j) \in E$, let

$$0 \leq \delta_{ij} \leq p_i, \quad \ell_{ij} = p_i - \delta_{ij} \in [0, p_i]. \quad (1)$$

The overlap allowance δ_{ij} does not shorten either task; it only reduces the amount of predecessor execution required before the successor may start. A feasible schedule satisfies

$$s_i \geq r_i \quad \forall i \in V, \quad (2)$$

$$s_j \geq s_i + \ell_{ij} \quad \forall (i, j) \in E, \quad (3)$$

and intervals assigned to one machine are disjoint:

$$[s_i, s_i + p_i) \cap [s_j, s_j + p_j) = \emptyset \quad \text{if } i \neq j \text{ and } \mu_i = \mu_j. \quad (4)$$

The objective is

$$C_{\max} = \max_{i \in V} (s_i + p_i). \quad (5)$$

Finish–start precedence is recovered by $\delta_{ij} = 0$; a zero lag is obtained by $\delta_{ij} = p_i$ and permits simultaneous starts on different machines.

For directed path $P = (v_1, \dots, v_k)$, including a singleton, define

$$Q_\delta(P) = p_{v_k} + \sum_{h=1}^{k-1} \ell_{v_h v_{h+1}} \quad (6)$$

$$= \sum_{h=1}^k p_{v_h} - \sum_{h=1}^{k-1} \delta_{v_h v_{h+1}},$$

$$L_\delta = \max_P Q_\delta(P). \quad (7)$$

For release dates, define

$$Q_{r,\delta}(P) = r_{v_1} + Q_\delta(P), \quad L_{r,\delta} = \max_P Q_{r,\delta}(P). \quad (8)$$

Lemma 1 (Workload and path lower bounds). *With $W = \sum_i p_i$, every feasible schedule satisfies*

$$C_{\max}^{\text{OPT}} \geq \max \left\{ \frac{W}{m}, L_{r,\delta} \right\}. \quad (9)$$

When all releases are zero, the second term is L_δ .

Proof. The machines supply at most $m C_{\max}$ units of processing capacity, so $C_{\max} \geq W/m$. For any path $P = (v_1, \dots, v_k)$, repeated application of (2)–(3) gives

$$s_{v_k} + p_{v_k} \geq r_{v_1} + \sum_{h=1}^{k-1} \ell_{v_h v_{h+1}} + p_{v_k} = Q_{r,\delta}(P).$$

Maximizing over paths proves the claim. $\square$

A. BOS Priority

For every task i , define its overlap-adjusted bottom level by

$$b_i = \max \left\{ p_i, \max_{(i,j) \in E} (\ell_{ij} + b_j) \right\}, \quad (10)$$

where the inner maximum is omitted for a sink. The explicit singleton term p_i is necessary: with a large overlap allowance, a continuation can be shorter than task i itself.

Lemma 2 (Bottom-level dynamic program). *For every task i ,*

$$b_i = \max_{P: P \text{ starts at } i} Q_\delta(P). \quad (11)$$

Thus $\max_i b_i = L_\delta$, and all scores are computed in $O(|V| + |E|)$ time by a reverse topological scan.

Proof. For a sink, the only path starting at i is (i) , so both sides are p_i . Assume the statement holds for each successor j of i . A path beginning at i is either (i) or edge (i, j) followed by a path beginning at j . Its best length is therefore exactly the right-hand side of (10) by the induction hypothesis. Each vertex and edge is inspected once. $\square$

For a fixed offline instance, b_i is static. BOS is event-driven because the executable set changes, not because unchanged scores are recomputed. Online changes to G , p , r , or δ lie outside the model and guarantees in this paper.

Algorithm 1 Event-driven BOS

Input: DAG $G = (V, E)$, p , r , δ , and m **Output:** start times s_i and assignments μ_i

```

1: Compute  $\ell_{ij}$  and  $b_i$  in reverse topological order
2:  $N \leftarrow V$ ;  $A, C \leftarrow \emptyset$ ; initialize idle machines
3: Put every release event into a minimum-time event queue  $Q$ 
4: while  $C \neq V$  do
5:    $t \leftarrow$  smallest timestamp in  $Q$ 
6:   Process all release, activation, and completion events at  $t$ 
7:   Insert each newly executable task in a max-heap keyed by  $b_i$ 
8:   while an idle machine and an executable task both exist do
9:     Extract a maximum-key task  $i$  and assign an idle machine
10:    Set  $s_i \leftarrow t$ ; move  $i$  from  $N$  to  $A$ 
11:    Enqueue completion at  $t + p_i$ 
12:    Enqueue activation at  $t + \ell_{ij}$  for every  $(i, j) \in E$ 
13:    Close all newly generated events stamped  $t$  and update the heap
14:   end while
15: end while

```

IV. EVENT-DRIVEN BOS

At time t , partition tasks into not-started, running, and completed sets $N(t)$, $A(t)$, and $C(t)$. After all events stamped t are closed, the executable set is

$$R(t) = \{i \in N(t) : t \geq r_i, j \in A(t) \cup C(t) \forall j \in \text{pred}(i), t \geq s_j + \ell_{ji} \forall j \in \text{pred}(i)\}. \quad (12)$$

BOS chooses a task of maximum b_i from $R(t)$, breaking ties by a fixed task index. It is work-conserving: after equal-time events have been closed, an idle machine is never left unused while $R(t)$ is nonempty.

There are three event types: release at r_i , edge activation at $s_i + \ell_{ij}$, and completion at $s_i + p_i$. Starting a task can create zero-lag activations at the current timestamp. These events must be processed to closure before advancing time.

An implementation stores one release flag and an activated-incoming edge counter per task. It therefore updates only endpoints affected by an event rather than rescanning the DAG.

Theorem 3 (Feasibility and single execution). *Algorithm 1 starts every task exactly once and returns a schedule satisfying (2)–(4).*

Proof. A task enters the ready heap only while it is in N and is removed from N when started, so it cannot be started twice. Dispatch occurs only after the task's release and all incoming activation events, which proves (2) and (3). A task is assigned only to an idle machine, which remains occupied until its completion event, proving (4). Zero-lag closure can start a successor at the same timestamp as its predecessor, but only after the predecessor start has been recorded, so the argument still applies. $\square$

Lemma 4 (No deadlock and termination). *Algorithm 1 terminates after finitely many events.*

Proof. Every start removes one task from N , so there are at most $|V|$ starts. If $N \neq \emptyset$ and all machines are occupied, a running task has a finite next completion. Otherwise, take

a source v in the induced DAG $G[N]$. Every predecessor of v has already started. If v is not executable, a finite release or activation event remains in the future; if neither does, v is executable and work conservation dispatches it. Equal-time events are exhausted before a strict time advance. There are $|V|$ releases, $|E|$ activations, and $|V|$ completions, excluding an infinite event sequence. $\square$

Proposition 5 (Implementation complexity). *With adjacency lists, counters, an event heap, and a ready heap, BOS runs in*

$$O((|V| + |E|) \log(|V| + |E|)) \quad (13)$$

time and $O(|V| + |E|)$ space.

Proof. Topological sorting and score computation are linear. Every task generates one release, start, and completion; every edge generates one activation. Each event and task enters a heap only a constant number of times. Graph storage, counters, state, and queues are linear. $\square$

V. APPROXIMATION ANALYSIS

The following argument is deliberately policy-agnostic. It uses work conservation, but never the ordering induced by b_i . It therefore certifies BOS and every other work-conserving list rule under the same model; the particular BOS priority must be judged empirically.

For a completed schedule S , let $q(t)$ be the number of busy machines and define the non-full times and total idle machine-time as

$$\mathcal{T}_{\text{nf}} = \{t \in [0, C_{\max}) : q(t) < m\}, \quad (14)$$

$$I = \int_0^{C_{\max}} (m - q(t)) dt. \quad (15)$$

The integral is only accounting notation: the state is constant between a finite number of events.

For task v in the realized schedule, put

$$a(v) = \max(\{r_v\} \cup \{s_u + \ell_{uv} : (u, v) \in E\}). \quad (16)$$

Thus $a(v) \leq s_v$. Choose a last-finishing task v_k . Working backward from v_h , stop if r_{v_h} attains $a(v_h)$, selecting the release term in a tie. Otherwise choose a predecessor v_{h-1} satisfying

$$a(v_h) = s_{v_{h-1}} + \ell_{v_{h-1}v_h} \quad (17)$$

and continue. Acyclicity makes this process terminate and produces one directed path $P = (v_1, \dots, v_k)$. Let $\rho = r_{v_1}$ and $Q = Q_\delta(P)$.

Lemma 6 (Single blocking-chain cover). *Up to the measure-zero event instants, every non-full time belongs to*

$$\mathcal{T}_{\text{nf}} \subseteq [0, \rho) \cup \bigcup_{h=1}^{k-1} [s_{v_h}, s_{v_h} + \ell_{v_h v_{h+1}}) \cup [s_{v_k}, s_{v_k} + p_{v_k}). \quad (18)$$

The intervals after $[0, \rho)$ are pairwise disjoint, have total length Q , and each contains a running chain task. Consequently,

$$I \leq m\rho + (m-1)Q. \quad (19)$$

Proof. Take $t \in \mathcal{T}_{\text{nf}}$. If $t \geq s_{v_k}$, then t is before the makespan and lies in v_k 's execution interval. Otherwise $t < s_{v_k}$. If $t \geq a(v_k)$, task v_k would be executable while a machine is idle, contradicting work conservation; hence $t < a(v_k)$.

If the chain step into v_h uses predecessor v_{h-1} , then $t < a(v_h) = s_{v_{h-1}} + \ell_{v_{h-1}v_h}$. Either $t \geq s_{v_{h-1}}$, placing t in that predecessor prefix, or $t < s_{v_{h-1}}$ and the same argument moves one step backward. If the recursion reaches v_1 without covering t , work conservation gives $t < a(v_1) = r_{v_1} = \rho$. This proves (18).

For consecutive chain tasks, $s_{v_{h+1}} \geq s_{v_h} + \ell_{v_h v_{h+1}}$, so the charged prefixes are ordered and disjoint; the final execution interval starts after the last prefix ends. Their total length is $\sum_{h < k} \ell_{v_h v_{h+1}} + p_{v_k} = Q$. Because $\ell_{v_h v_{h+1}} \leq p_{v_h}$, task v_h runs throughout its charged prefix. Thus at most $m-1$ machines are idle on charged time after ρ , while at most m are idle before ρ . Integration proves (19). $\square$

Theorem 7 (Zero-release bound). *If every $r_i = 0$, any work-conserving list schedule S satisfies*

$$C_{\max}^S \leq \frac{W}{m} + \left(1 - \frac{1}{m}\right) L_\delta \leq \left(2 - \frac{1}{m}\right) C_{\max}^{\text{OPT}}. \quad (20)$$

Proof. Lemma 6 has $\rho = 0$ and $Q \leq L_\delta$, hence $I \leq (m-1)L_\delta$. Capacity accounting gives $mC_{\max}^S = W + I$, which proves the first inequality. Both W/m and L_δ lower-bound the optimum by Lemma 1; applying the two bounds separately gives the ratio. $\square$

Theorem 8 (Release-aware bound). *For arbitrary finite releases, any work-conserving list schedule S satisfies*

$$C_{\max}^S \leq \frac{W}{m} + L_{r,\delta} \leq 2C_{\max}^{\text{OPT}}. \quad (21)$$

Proof. By capacity accounting and (19),

$$\begin{aligned} C_{\max}^S &= \frac{W + I}{m} \\ &\leq \frac{W}{m} + \rho + \left(1 - \frac{1}{m}\right) Q \\ &\leq \frac{W}{m} + \rho + Q. \end{aligned}$$

Because P starts at v_1 , $\rho + Q = Q_{r,\delta}(P) \leq L_{r,\delta}$. Lemma 1 then yields the factor two. $\square$

The release-free expression cannot simply be reused with releases: one unit-time task released at time R finishes at $R+1$, whereas a formula containing only W/m and L_δ is independent of R . More broadly, the analysis assumes identical machines, non-preemption, a fixed DAG, finite releases, $0 \leq \ell_{ij} \leq p_i$, and work-conserving dispatch. It does not cover heterogeneous execution times, setup times, uncertain data, or online graph changes.

VI. SCHEDULE-INDUCED BOTTLENECK DIAGNOSIS

The input DAG records temporal precedence but cannot expose delay caused by competition for a machine. Given a completed work-conserving schedule S , add a virtual source 0 and an arc between each consecutive task pair on each realized machine sequence. Formally, use the labeled arc multiset

$$\mathcal{A}_S = E_r \uplus E_P \uplus E_M, \quad (22)$$

where release arc $(0, i, r_i) \in E_r$, precedence arc $(i, j, \ell_{ij}) \in E_P$, and machine-order arc $(i, j, p_i) \in E_M$ all encode $s_j \geq s_i + w_{ij}$ with $s_0 = 0$. The multiset notation is intentional: a precedence and a machine arc may have the same endpoints but different weights and meanings.

Lemma 9 (Augmented-DAG certificate). *The labeled graph $G_S^+ = (V \cup \{0\}, \mathcal{A}_S)$ is acyclic. For a schedule returned by a work-conserving event-driven list rule, its longest-path distances from 0 equal the realized task start times. Therefore,*

$$C_{\max}^S = \max_{i \in V} (d_i + p_i), \quad (23)$$

and a reconstructed longest path is a critical-chain certificate for the fixed machine order.

Proof. The original precedence graph is acyclic and release arcs leave only the source. Any cycle would therefore contain a machine-order arc of positive weight. Summing all encoded inequalities around it would imply $s_i \geq s_i + \varepsilon$ for some $\varepsilon > 0$, a contradiction.

All incoming arcs give $d_i \leq s_i$. Conversely, consider tasks in realized start order. At s_i , either an incoming release/precedence constraint is tight or the preceding task on i 's assigned machine completes. If none were tight, that machine would have been idle and i executable immediately before s_i , contradicting work conservation (event instants are handled by same-time closure). Induction therefore gives $d_i = s_i$, and (23) follows. $\square$

For any labeled arc (u, v, w) , its nonnegative slack is

$$\sigma_{uv} = s_v - s_u - w. \quad (24)$$

The implementation reports tight release, precedence, and machine arcs, reconstructs one critical chain, and verifies (23). This is a post-schedule explanation: it can show that two priority rules induced different resource bottlenecks, but it does not turn the fixed BOS score into a dynamic score or prove that one priority is globally preferable.

VII. EXPERIMENTAL DESIGN

The evaluation asks four questions: (RQ1) does every implementation return a feasible schedule and satisfy the policy-independent proved bound; (RQ2) does an overlap-aware path priority improve makespan relative to path-, workload-, duration-, and arrival-based list rules; (RQ3) how far are the heuristics from certified optima on small instances; and (RQ4) what mechanisms and runtime costs accompany the observed differences?

TABLE I
PRIORITY RULES INSIDE THE COMMON EVENT SIMULATOR.

Policy	Descending priority (except FIFO)
BOS	b_i from (10); overlap-adjusted longest path.
HEFT-rank	$h_i = p_i + \max_{(i,j) \in E} h_j$; full-work downstream path.
GRPW	$p_i + \sum_{j \in \text{Desc}(i)} p_j$ over distinct transitive successors.
LPT	Larger p_i first.
SPT	Smaller p_i first.
FIFO	Earlier entry into the executable set first.

A. Controlled Priority Comparators

Every comparator uses the same event queue, executable-set definition, machine-selection rule, and task-index tie break. Only the priority key is changed. This control is important because full HEFT also changes processor selection and targets heterogeneous processors [5].

GRPW, LPT, and SPT are standard project-scheduling priority-rule patterns; we use the all-successor GRPW definition shown in Table I and state it explicitly because names vary across implementations [6]. HEFT-rank is not labeled “HEFT”: it retains only the upward-rank idea while deliberately removing heterogeneity and communication.

B. Synthetic Main Battery

Processing times are independent discrete uniform values in $\{1, \dots, 20\}$. Four deterministic generator families cover a deep backbone with short detours (chain-heavy), repeated fork-join modules, layered many-to-one merges (merge-heavy), and a random layered DAG. The generator source records every edge and all actual overlap fractions.

For target overlap $\alpha \in \{0, .25, .50, .75\}$, $\alpha = 0$ sets every $\delta_{ij} = 0$. Otherwise, independently by edge,

$$X_{ij} \sim \text{Beta}(14\alpha, 14(1-\alpha)), \quad \delta_{ij} = \text{round}(p_i X_{ij}). \quad (25)$$

The two release policies are $r_i = 0$ and $r_i \sim \text{DU}\{0, \dots, \text{round}(.5W/m)\}$.

The full factorial design uses $n \in \{50, 100, 250\}$, $m \in \{2, 4, 8\}$, four overlap targets, two release policies, four families, and 20 seeds. It contains 5,760 instances and 34,560 heuristic schedules. The chain-heavy generator is a forced-backbone sanity family on which the six rules coincide. Its rows remain in the raw audit and bound checks, but comparative plots and tables exclude it so that structural ties do not dilute differences. Likewise, $\alpha = 0$ remains a sanity condition—BOS and HEFT-rank are algebraically identical there—but paired priority comparisons report positive-overlap instances. This leaves 1,620 paired instances per release condition.

C. Certified Small Instances and Stress Tests

For ground truth, the exact battery crosses the four families, $n \in \{8, 10\}$, $m \in \{2, 3\}$, $\alpha \in \{0, .50, .80\}$, both release policies, and six seeds. A time-indexed MILP minimizes makespan

with one binary start-time variable per task and integer time, capacity, release, and lag constraints. SciPy’s `milp` interface to HiGHS solves the model [11]; all 576 instances terminated with status optimal and zero reported MIP gap under a 20-second limit. Comparative exact statistics exclude the 144 forced chain-heavy cases, leaving 432 instances. Appendix A states the formulation.

A separate contrast search generates 4,000 instances with $n = 12$, $m = 3$, overlap targets .25, .50, .75, and .90, then exactly solves the five largest BOS wins and losses against HEFT-rank. The scaling study times BOS and HEFT-rank on two families at $n \in \{100, 500, 1000, 2500, 5000\}$ with eight repetitions. These two rules share linear-time score recurrences; GRPW’s explicit transitive-successor sets are intentionally omitted from this like-for-like runtime comparison. Heuristic timing uses one Python process in a Linux container on an AMD EPYC 9V74 virtual CPU. The recorded environment is Python 3.12.14, NumPy 2.3.5, pandas 2.2.3, SciPy 1.17.0, and Matplotlib 3.10.8.

D. Metrics and Statistical Reporting

The analytical lower bound is $LB = \max\{W/m, L_{r,\delta}\}$. For comparator X , paired BOS advantage is

$$A_X = 100 \frac{C_{\max}^X - C_{\max}^{\text{BOS}}}{C_{\max}^X}, \quad (26)$$

so positive values favor BOS. We report its mean, win/tie/loss counts, and a 95% cluster-bootstrap interval with 5,000 resamples of family-seed clusters. The exact gap is $100(C_{\max} - C^*)/C^*$ and an optimum hit requires exact equality.

Once BOS and HEFT-rank diverge, they no longer occupy the same run-time state. We therefore do not compare “same-state decisions.” Instead, we collect every unique candidate set of size at least two seen in either trace and evaluate both deterministic priorities offline on each common set. The reported disagreement fraction measures the scoring mechanism without pretending that the two schedules share a state trajectory.

VIII. RESULTS

A. Feasibility and Bound Audit (RQ1)

All 34,560 main-battery schedules passed independent checks for releases, every incoming lag, non-overlap on each assigned machine, single execution, and makespan reconstruction. None violated its applicable theorem bound. This is a consistency audit rather than evidence that BOS alone owns the guarantee: all six policies are work-conserving and Theorems 7–8 apply to all of them.

B. Aggregate Heuristic Comparison (RQ2)

Table II shows two regimes. Against simple duration or arrival rules, the difference is visible: with zero releases, mean BOS advantages are 1.58% over LPT, 2.98% over SPT, and 1.82% over FIFO. Against structure-aware priorities, however, the effect is small: 0.188% over HEFT-rank and 0.283% over GRPW. Uniform releases reduce these means to 0.140% and

TABLE II

PAIRED BOS ADVANTAGE (%) ON POSITIVE-OVERLAP, NONTRIVIAL-FAMILY INSTANCES. POSITIVE MEANS BOS HAS SMALLER MAKESPAN; INTERVALS USE THE FAMILY-SEED CLUSTER BOOTSTRAP.

Releases	Comparator	Mean	95% CI	Win	Tie	Loss	N
Zero	HEFT-rank	0.19%	[0.13, 0.25]	551	807	262	1620
Zero	GRPW	0.28%	[0.22, 0.35]	593	763	264	1620
Zero	LPT	1.58%	[1.34, 1.82]	1202	363	55	1620
Zero	SPT	2.98%	[2.75, 3.19]	1436	151	33	1620
Zero	FIFO	1.82%	[1.66, 1.99]	1329	226	65	1620
Uniform	HEFT-rank	0.14%	[0.09, 0.19]	552	795	273	1620
Uniform	GRPW	0.19%	[0.14, 0.24]	561	806	253	1620
Uniform	LPT	1.13%	[0.96, 1.31]	1165	400	55	1620
Uniform	SPT	2.12%	[1.96, 2.28]	1430	156	34	1620
Uniform	FIFO	1.32%	[1.20, 1.44]	1309	256	55	1620

TABLE III

EXACT COMPARISON ON 432 NONTRIVIAL INSTANCES.

Policy	Mean gap	Max gap	Optimum hits
BOS	1.78%	17.33%	61.6%
HEFT-rank	1.81%	17.33%	63.0%
GRPW	1.80%	15.56%	62.0%
LPT	2.58%	27.59%	55.3%
SPT	6.22%	34.48%	25.7%
FIFO	3.98%	28.95%	41.9%

0.191%, respectively. Every cluster-bootstrap interval is above zero, but the instance counts rule out dominance. For example, in the zero-release BOS-HEFT comparison, BOS wins 551 cases, ties 807, and loses 262; under uniform releases the counts are 552, 795, and 273.

Across positive-overlap nontrivial cases, mean $C_{\max}/LB$ under zero releases is 1.0256 for BOS, 1.0277 for HEFT-rank, 1.0287 for GRPW, 1.0427 for LPT, 1.0585 for SPT, and 1.0454 for FIFO. With uniform releases the corresponding ratios are 1.2488, 1.2505, 1.2511, 1.2636, 1.2761, and 1.2656. Figure 1 shows the full overlap trend.

The priority mechanisms increasingly diverge even though aggregate makespan changes remain modest. As target overlap rises from .25 to .50 and .75, common-candidate disagreement rises from 28.3% to 38.9% and 49.2% for zero releases; the uniform-release values are 26.3%, 37.3%, and 47.5% (Fig. 2). Thus replacing the score changes many local choices, but downstream slack and parallel capacity often absorb those changes.

C. Certified Optimality (RQ3)

BOS has the smallest mean exact gap, 1.783%, but only narrowly: GRPW has 1.803% and HEFT-rank 1.809%. Other metrics reverse the ordering. HEFT-rank has the highest optimum-hit rate at 63.0%, versus 62.0% for GRPW and 61.6% for BOS; GRPW also has the smallest maximum observed gap, 15.56%, versus 17.33% for BOS and HEFT-rank. The exact results support an average-case advantage for BOS under this generator, not a pointwise or distribution-free superiority claim.

D. Mechanism Contrasts and Runtime (RQ4)

The largest certified BOS win is a zero-release, merge-heavy instance at target overlap .75: BOS obtains $C_{\max} = 38 = C^*$ and HEFT-rank obtains 44, a 13.64% paired advantage. The stored Gantt chart shows this schedule alongside its exact replay data. The search also finds the opposite extreme. On a zero-release random-layered instance at target overlap .90, BOS obtains 41 while HEFT-rank obtains $35 = C^*$, a 17.14% BOS loss. The stored instance JSON and augmented-DAG diagnostics make both counterexamples exactly replayable.

Median BOS time grows from 0.592 ms at $n = 100$ to 68.637 ms at $n = 5000$; HEFT-rank grows from 0.552 ms to 71.684 ms. Fitted log-log slopes are 1.201 and 1.232. These measurements are consistent with low-order event-driven overhead, but they are implementation measurements rather than a substitute for the worst-case complexity in Proposition 5.

IX. CONCLUSION AND LIMITATIONS

BOS is a deterministic event-driven list scheduler for identical machines with releases and edge-specific minimum start-start lags. Its priority is the longest overlap-adjusted downstream path, while feasibility is enforced by explicit release, activation, and completion events. A single realized blocking chain proves a $(2 - 1/m)$ guarantee without releases and a factor-two guarantee with arbitrary finite releases. The same proof also marks the boundary of the theoretical claim: it follows from work conservation and does not prove that BOS is better than another list priority.

The expanded experiments compare six deterministic policies rather than using HEFT-rank as the sole foil. BOS has the lowest mean makespan against all five comparators in the reported positive-overlap groups and the smallest mean gap on 432 nontrivial exact instances. The margins against HEFT-rank and GRPW are small, while their hit-rate and worst-case summaries reverse parts of the ranking. Certified examples in both directions further reject a dominance interpretation.

The evidence is limited to synthetic DAGs, integer processing data, one release distribution, and exact instances of at most ten tasks. The cluster-bootstrap intervals quantify variation within this generator, not external validity for a factory or project portfolio. Setup times, machine eligibility,

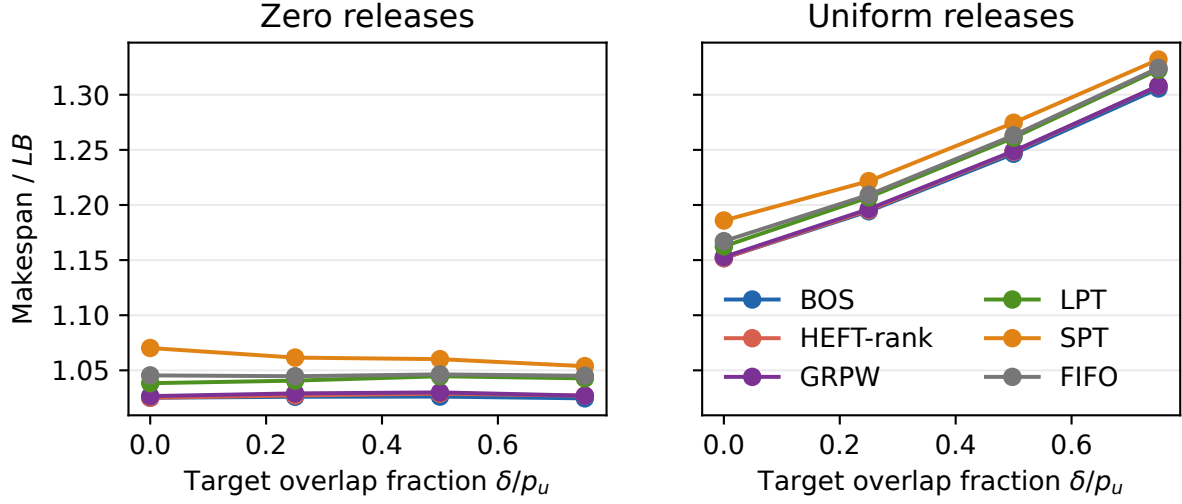


Fig. 1. Mean makespan normalized by the analytical lower bound, excluding the forced chain-heavy sanity family. The growing release-case ratio mainly reflects that LB does not capture all interactions between release idle time and resource competition.

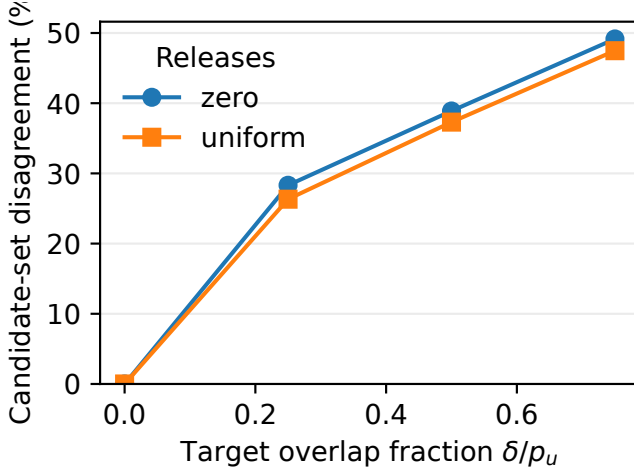


Fig. 2. BOS–HEFT-rank disagreement on common candidate sets, excluding the forced chain-heavy family.

heterogeneous speeds, stochastic durations, online arrivals, and OR-type material choices are outside the present model. Promising next steps are public RCPSP/max-derived benchmarks, industrial trace calibration, richer lower bounds, and adaptive or learned priorities whose feasibility layer and evaluation budget remain controlled.

APPENDIX A EXACT FORMULATION AND REPRODUCTION

A. Time-Indexed MILP

Let H be the best makespan returned by the six list rules and let binary x_{it} indicate that task i starts at integer time $t \in \{r_i, \dots, H - p_i\}$. With continuous variable C , the exact

model is

$$\min C, \quad (27)$$

$$\text{s.t.} \quad \sum_t x_{it} = 1 \quad \forall i, \quad (28)$$

$$\sum_i \sum_{t: t \leq \tau < t + p_i} x_{it} \leq m \quad \forall \tau = 0, \dots, H - 1, \quad (29)$$

$$\sum_t t x_{jt} - \sum_t t x_{it} \geq \ell_{ij} \quad \forall (i, j) \in E, \quad (30)$$

$$C \geq \sum_t t x_{it} + p_i \quad \forall i. \quad (31)$$

Release dates are enforced by each variable’s time domain. For fixed integer intervals, the capacity constraints bound the maximum clique of the associated interval graph by m ; interval-graph coloring therefore assigns the intervals to m identical machines. The implementation accepts a run as certified only when the solver reports success, status optimal, decoded feasibility, and zero MIP gap.

B. Executable Workflow

The repository separates model logic, deterministic generators, exact optimization, experiment orchestration, tests, raw CSVs, figures, and this manuscript. From the project directory, the complete workflow is

```
python -m pip install -r requirements.txt
PYTHONPATH=. python -m unittest \
    discover -s tests -v
PYTHONPATH=. python -m \
    experiments.run_experiments \
    --profile paper --all
latexmk -pdf \
    BOS_greedy_20260915.tex
```

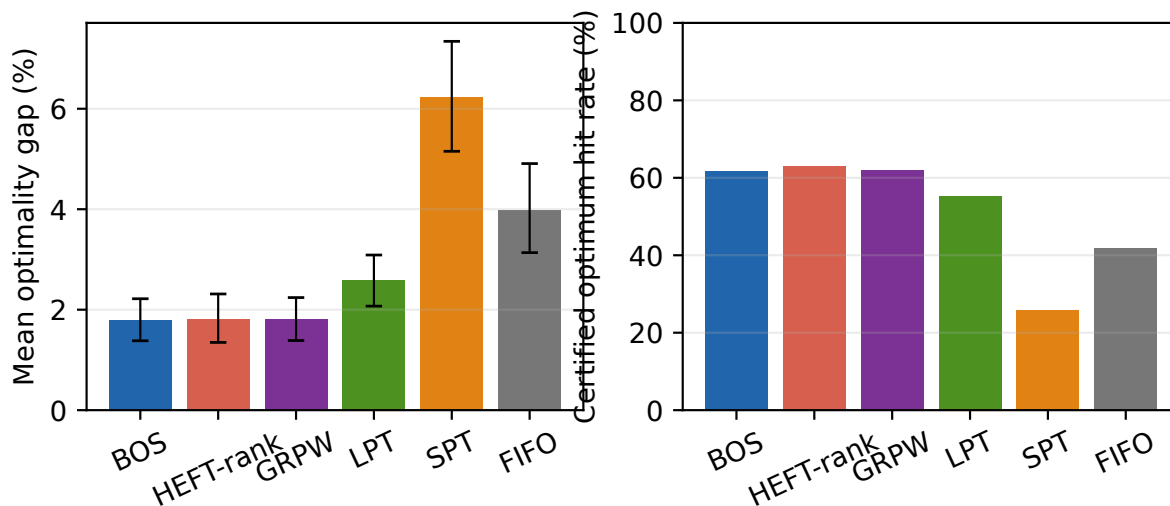


Fig. 3. Mean optimality gap with cluster-bootstrap intervals and certified optimum hit rate. Forced chain-heavy ties are excluded.

The paper run writes checkpoints after each phase. The raw files retain the forced chain-heavy sanity cases and zero-overlap controls even where the comparative manuscript summaries exclude them. Seeds, generator coordinates, per-policy makespans, lower bounds, run times, exact solver gaps, and selected counterexample instances are all stored rather than inferred from plots.

REFERENCES

- [1] M. L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*, 5th ed. Cham, Switzerland: Springer, 2016.
- [2] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, Mar. 1969.
- [3] M. Bartusch, R. H. Möhring, and F. J. Radermacher, "Scheduling project networks with resource constraints and time windows," *Annals of Operations Research*, vol. 16, pp. 199–240, 1988.
- [4] S. Hartmann and D. Briskorn, "A survey of variants and extensions of the resource-constrained project scheduling problem," *European Journal of Operational Research*, vol. 207, no. 1, pp. 1–14, 2010.
- [5] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
- [6] R. Kolisch, "Serial and parallel resource-constrained project scheduling methods revisited: Theory and computation," *European Journal of Operational Research*, vol. 90, no. 2, pp. 320–333, 1996.
- [7] A. Schutt, T. Feydy, P. J. Stuckey, and M. G. Wallace, "Solving RCPSP/max by lazy clause generation," *Journal of Scheduling*, vol. 16, no. 3, pp. 273–289, 2013.
- [8] P. Brucker, A. Drexler, R. H. Möhring, K. Neumann, and E. Pesch, "Resource-constrained project scheduling: Notation, classification, models, and methods," *European Journal of Operational Research*, vol. 112, no. 1, pp. 3–41, 1999.
- [9] S. Hartmann and D. Briskorn, "An updated survey of variants and extensions of the resource-constrained project scheduling problem," *European Journal of Operational Research*, vol. 297, no. 1, pp. 1–14, 2022.
- [10] C. Schwindt, *Resource Allocation in Project Management*, ser. Lecture Notes in Economics and Mathematical Systems. Berlin, Germany: Springer, 2005, vol. 559.
- [11] P. Virtanen, R. Gommers, T. E. Oliphant *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.